

CONCEPT NOTE

Project Title:

AI-Powered Career Opportunity Filtering System

1. Background/Context

In today's digital environment, students and job seekers receive a large number of job and internship opportunities through emails and online platforms. However, most of these opportunities are unstructured, repetitive, and often not aligned with individual preferences or qualifications. This leads to significant time consumption and confusion in identifying relevant opportunities.

There is currently a lack of automated systems that can intelligently analyze incoming opportunities, filter them based on user-specific criteria, and present structured outputs. Addressing this gap is important to improve efficiency, reduce manual effort, and support better career decision-making for students and fresh graduates.

2. Objectives

The primary objective of this project is to develop an AI-powered system that automates the process of identifying, analyzing, and filtering job and internship opportunities from emails.

The project aims to:

- Extract meaningful information from unstructured email content
- Identify whether the content is job/internship-related
- Match opportunities with user preferences such as role, salary, and eligibility
- Store structured and categorized data for easy access and tracking

3. Target Audience/Beneficiaries

The primary beneficiaries of this project are:

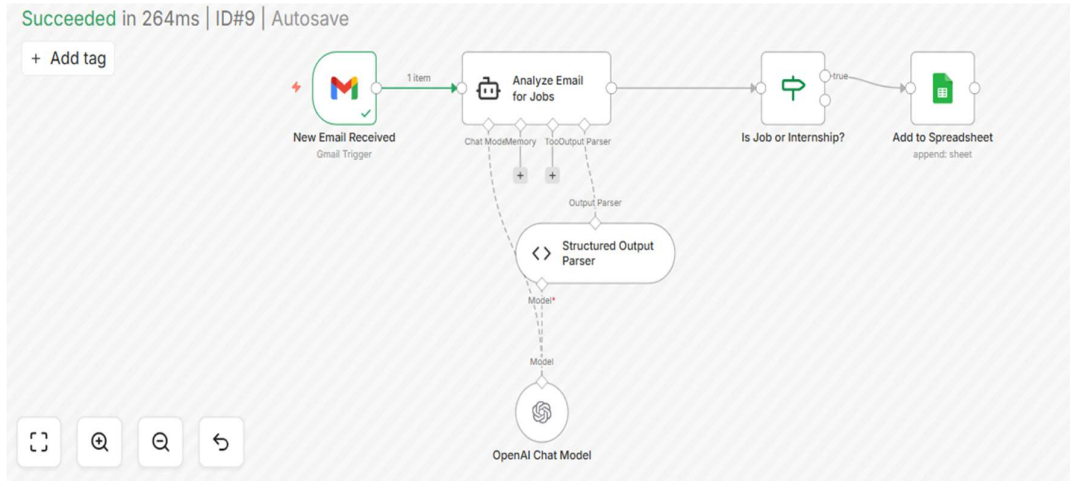
- College students actively searching for jobs or internships
- Fresh graduates looking for entry-level opportunities

Secondary beneficiaries include:

- Individuals receiving frequent job alerts via email
- Users seeking automated tools to improve productivity in job search

4. Key Activities

1. Designed the initial workflow architecture
2. Implemented Gmail Trigger to capture incoming emails automatically



3. Developed AI Agent 1 for email analysis using OpenAI Chat Model

Prompt used for extracting job-related information

Prompt:

You are an expert system for analyzing email content to identify job or internship opportunities.

Your task is to extract structured job data **ONLY** if a **REAL** opportunity exists.

Note: If the mail is not related to JOB or internship, place null values in the output

STRICT RULES:

1. Classify as job/internship **ONLY IF**:
 - Hiring is explicitly mentioned
 - A role/title is present
 - **AND** an application link **OR** clear apply instruction is available
2. Ignore emails that are:
 - Promotional (e.g., "Top jobs for you", "Upgrade your skills")
 - Newsletters without direct application
 - Courses, webinars, events
3. If multiple job opportunities are present:
 - Extract **ONLY** the top 1–2 most relevant roles
 - Prefer roles that are clearly defined and fresher-friendly
4. **DO NOT GUESS** any field
 - If information is missing → return null

EXTRACTION FIELDS:

- isJobOrInternship → true / false

- company → string or null
- role → string or null
- salary →
Convert into:
 - "X LPA" (for jobs)
 - "₹X/month" (for internships)
 - If not mentioned → null
- link → valid application link or null
- platformOrUser →
Identify source:
 - Internshala / LinkedIn / Unstop / Naukri / Company / Other
- deadline →
Format: YYYY-MM-DD
If not available → null
- offerType → "Job" or "Internship" or null

OUTPUT FORMAT (STRICT JSON ONLY):

```
{
  "isJobOrInternship": "",
  "company": "",
  "role": "",
  "salary": "",
  "link": "",
  "platformOrUser": "",
  "deadline": "",
  "offerType": ""
}
```

The screenshot displays the 'Analyze Email for Jobs' application interface. It is divided into three main sections: INPUT, PARAMETERS, and OUTPUT.

INPUT Section: Shows a single email item with the following details:

- id:** 19d6cb90d5f07970
- threadid:** 19d6cb90d5f07970
- labels:**
 - labels[0]: IMPORTANT
 - labels[1]: CATEGORY_PERSONAL
 - labels[2]: INBOX
- sizeEstimate:** 7977
- headers:**
 - delivered-to: Delivered-To: newtripti925@gmail.com
 - received: Received: from mail-sor-f41.google.com (mail-sor-f41.google.com [209.85.220.41]) \n by mx.google.com with SMTPS id 650f5600204a3-6503a9a0763sor9167969d50.10.2026.04.08.03.52.39 \n for <newtripti925@gmail.com> \n (Google Transport Security); \n Wed, 08 Apr 2026 03:52:40 -0700

PARAMETERS Section: Contains configuration for the analysis process.

- Source for Prompt (User Message):** Define below
- Prompt (User Message):** /x {{ \$json.text }}
Hello Tripti this is to inform you that , we re-
- Require Specific Output Format:** ☒
- Enable Fallback Model:** ☐
- Options:** Chat Model, Memory, Tool, Output Parser

OUTPUT Section: Displays the results of the analysis for 1 item.

- isJobOrInternship:** true
- company:** Narula
- salary:** 8 LPA
- link:** https://www.jobcliff.com/job-detail/510
- platformOrUser:** Other
- deadline:** 2026-04-15
- offerType:** Job

4. Integrated Structured Output Parser to standardize extracted data

The screenshot shows the 'Structured Output Parser' interface. On the left, under the 'INPUT' tab, there is a tree view of an email object. The 'headers' field is expanded, showing 'delivered-to', 'received', 'x-received', and 'arc-seal'. The 'received' field contains a detailed email header. On the right, the 'Schema Type' is set to 'Define using JSON Schema'. Below this, the 'Input Schema' is displayed as a JSON object with properties like 'isJobOrInternship', 'company', 'Role', 'salary', and 'link'. The schema is defined with types like 'object', 'boolean', and 'string'. There are also checkboxes for 'Auto-Fix Format' and 'Customize Retry Prompt'.

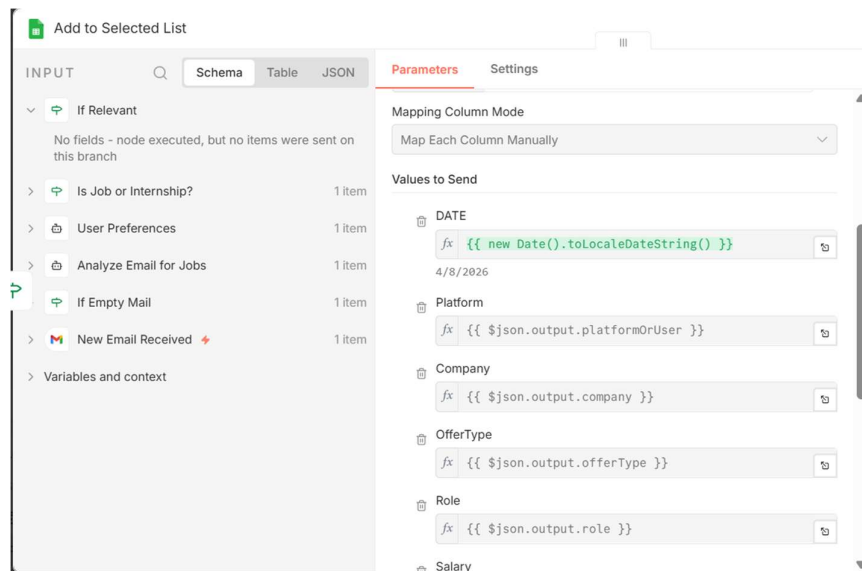
5. Applied IF condition to classify emails as job/internship or not

TRUE → Continue workflow

FALSE → Stored in separate sheet (non-job data)

The screenshot shows the 'Is Job or Internship?' workflow node. On the left, the 'INPUT' tab shows a tree view of a job listing object. The 'output' field is expanded, showing 'isJobOrInternship' (true), 'company' (Narula), 'role', 'salary' (8 LPA), 'link', 'platformOrUser' (Other), 'deadline' (2026-04-15), 'offerType' (Job), 'isRelevant' (false), 'reason' (Role is unspecified/unclear in the job data, so the listing is rejected despite salary and skill alignment), and 'match_score' (30). In the center, the 'Conditions' section shows a JSON path expression: `{{ $json.output.isJobOrInternship }}` with a dropdown set to 'is true'. There is also a checkbox for 'Convert types where required'. On the right, the 'OUTPUT' tab shows the 'True Branch (1 item)' and 'False Branch'. The 'True Branch' output is a tree view of the same job listing object as the input, with 'isJobOrInternship' set to true. The 'False Branch' is empty.

6. Created manual edit fields to organize and structure extracted data into columns



7. Added second AI Agent for relevance checking based on user profile

- Prompt includes user preferences (role, salary, eligibility)
- Input: Structured job data
- Output: Relevance decision (true/false), score, and reason

Prompt:

You are an AI assistant that evaluates job opportunities based on a user profile.
You are given structured job data extracted from an email.

INPUT DATA:

Job Data:

- isJobOrInternship: {{ \$json["output"]["isJobOrInternship"] }}
- company: {{ \$json["output"]["company"] }}
- role: {{ \$json["output"]["Role"] }}
- salary: {{ \$json["output"]["salary"] }}
- link: {{ \$json["output"]["link"] }}
- platformOrUser: {{ \$json["output"]["platformOrUser"] }}
- deadline: {{ \$json["output"]["deadline"] }}
- offerType: {{ \$json["output"]["offerType"] }}

USER PROFILE:

Resume:

[pasted my resume]

Preferences (User's Profile):

- Preferred Roles: Software Developer, Data Analyst
- Minimum Salary: 4 LPA
- Graduation Year: 2026

EVALUATION RULES:

1. Reject if:

- isJobOrInternship = false
- link is null
- role is unclear or too generic
- email is promotional or indirect

2. Role Matching:

- Prefer roles: Software Developer, Data Analyst
- Allow closely related roles (e.g., Backend Developer, ML Engineer, Data Science Intern)

SALARY & OFFER TYPE LOGIC:

A. For JOBS:

- If salary is provided:
 - Accept ONLY if salary $\geq$ 4 LPA
 - If salary $<$ 4 LPA → mark isRelevant = false
- If salary is null:
 - Do NOT reject, but reduce match_score

B. For INTERNSHIPS:

- Accept ALL internships (stipend or unpaid)
- Do NOT reject based on salary/stipend
- Still check role relevance

ADDITIONAL RULES:

3. Fresher Eligibility:

- Prefer roles suitable for 2026 graduates

4. Deadline Handling:

- If deadline is in the past → mark isRelevant = false

SCORING LOGIC:

- Start from 0
- Add based on:
 - Role match
 - Skill alignment (Python, ML, SQL, Web)
 - Fresher eligibility
 - Salary fit (ONLY for jobs)
- Final match_score: 0–100

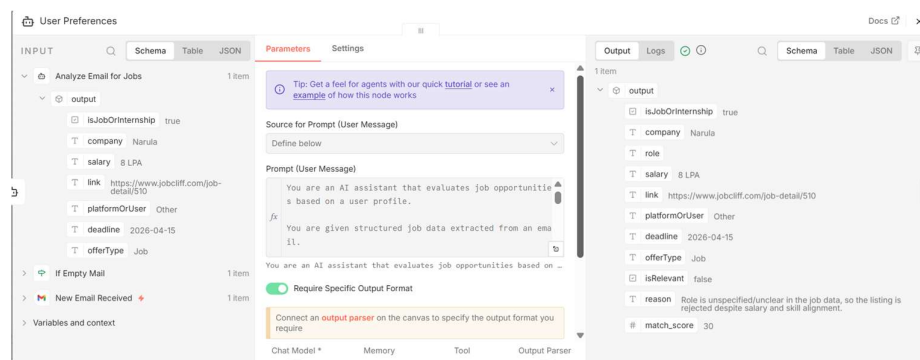
OUTPUT RULES:

- Return ONLY valid JSON

- Do NOT add extra text
- Keep explanation short and precise (1–2 lines)

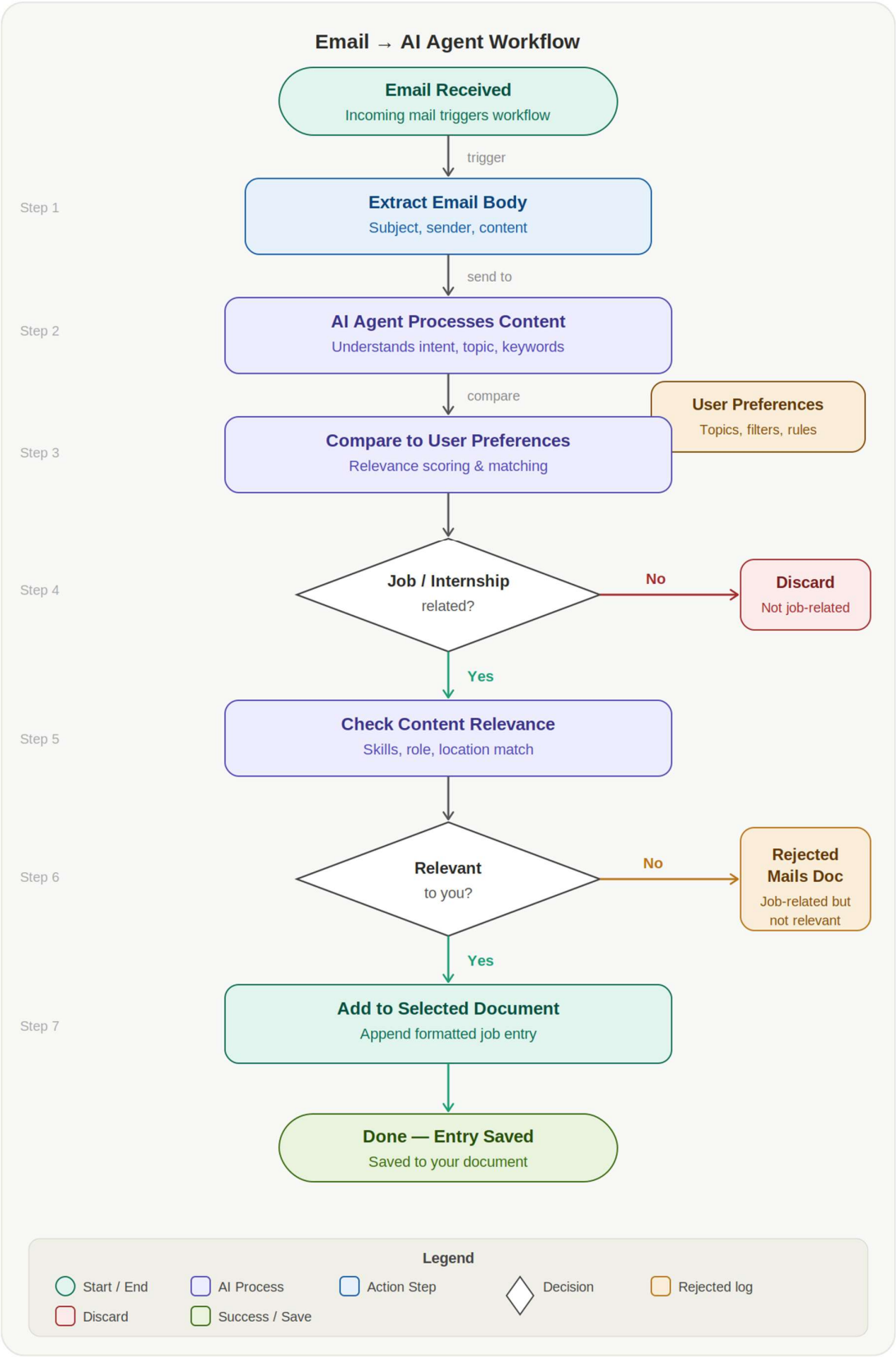
OUTPUT FORMAT:

```
{
  "isJobOrInternship": "",
  "company": "",
  "role": "",
  "salary": "",
  "link": "",
  "platformOrUser": "",
  "deadline": "",
  "offerType": "",
  "isRelevant": "",
  "reason": "",
  "match_score": ""
}
```



- Implemented decision logic for final classification
 - Relevant jobs → Stored in Selected Sheet
 - Non-relevant jobs → Stored in Rejected Sheet
- Created separate Google Sheets for structured data storage
 - Sheet 1: Selected Opportunities
 - Sheet 2: Rejected Opportunities (job-related but not relevant)
- Implemented handling for non-job emails: Directly stored in a separate sheet using FALSE condition
- Added edge case handling for empty emails: IF condition to detect empty input that prevents unnecessary processing
- Created a separate worksheet for edge test cases : Includes scenarios like missing data, multiple jobs, empty emails

Workflow



7. Budget Estimate

- OpenAI API Usage: ₹1,000 – ₹2,000 (based on testing and executions)
- Automation Platform (n8n): ₹0 (local setup) or minimal hosting cost
- Data Storage (Google Sheets): Free
- Development Cost: Self-developed (no external cost)

Total Estimated Budget: ₹1,000 – ₹2,500

8. Conclusion

The AI-Powered Career Opportunity Filtering System successfully demonstrates how automation and AI can be integrated to solve real-world problems in job search management. The project efficiently converts unstructured email data into structured, actionable insights while applying personalized filtering logic.

The implementation of dual AI agents, conditional workflows, and structured storage ensures both accuracy and usability. This system significantly reduces manual effort and enhances decision-making for users, making it a scalable and impactful solution.

9. Contact Information

Name: Tripti Kumari

Program: B.Tech in Computer Science Engineering

Institution: Narula Institute of Technology

Internship ID: 2026AICTECSRBOX1535

Email: vishwakarmap1305@gmail.com

LinkedIn: <https://www.linkedin.com/in/tripti-kumari-8b3abb253/>